

# Debugging di Base

---

## Debugging di Base

*Tecniche e strumenti per trovare e correggere gli errori nei programmi C++.*

## Cosa significa fare debugging?

Tutti i programmatori scrivono codice con bug. Non è una questione di bravura — è normale. La differenza tra un programmatore esperto e uno alle prime armi spesso sta nella **velocità con cui trovano e correggono** gli errori.

Il debugging (eliminazione dei bug) è un'abilità che si impara con la pratica. Queste tecniche ti aiuteranno a trovare i problemi più velocemente.

## I tre tipi di errori

- **Errori di compilazione:** il compilatore si rifiuta di creare il programma. Ti dice esattamente dove c'è il problema.
- **Errori di runtime:** il programma parte ma poi si blocca o crasha. Devi capire perché.
- **Errori logici:** il programma gira, non crasha, ma produce risultati sbagliati. I più subdoli.

## Leggere i messaggi di errore del compilatore

Quando il compilatore trova un errore, ti dice esattamente dove:

```
hello.cpp:5:5: error: 'cout' was not declared in this scope
  cout << "Ciao!" << endl;
  ^~~~
```

Il formato è: `file:riga:colonna: tipo: messaggio`

- `hello.cpp` — il file con l'errore
- `5` — la riga dell'errore
- `error` — tipo di problema ( `error` = grave, `warning` = avviso)
- Il messaggio spiega cosa c'è di sbagliato

**Consiglio importante:** correggi sempre il **primo** errore nella lista. Spesso un errore ne genera altri a cascata. Una volta sistemato il primo, molti degli altri spariscono.

## I warning: segnali di pericolo

I warning non impediscono la compilazione, ma segnalano codice potenzialmente problematico:

```
warning: unused variable 'x' [-Wunused-variable]
```

Non ignorarli — spesso indicano errori logici nascosti.

Compila sempre con `-Wall` per vedere tutti i warning:

```
g++ programma.cpp -o programma -Wall
```

## La tecnica più semplice: stampe di debug

Il modo più diretto per capire cosa fa il programma è aggiungere `cout` per stampare i valori delle variabili nei punti chiave:

```
double calcola(int a, int b) {  
    cout << "[DEBUG] a=" << a << ", b=" << b << endl; // stampa i valori  
    in entrata  
  
    double risultato = a / b; // possibile divisione intera!  
  
    cout << "[DEBUG] risultato=" << risultato << endl;  
  
    return risultato;  
}
```

Aggiungi il prefisso `[DEBUG]` così quando hai finito sai quali `cout` eliminare.

# Gli errori più comuni

## Variabile non inizializzata

```
int x;  
cout << x;    // stampa un numero casuale – bug!  
  
// Soluzione: inizializza sempre  
int x = 0;
```

## Divisione intera indesiderata

```
int a = 7, b = 2;  
double risultato = a / b;    // 3.0, non 3.5! (divisione intera)  
  
// Soluzione: converti uno dei due prima di dividere  
double risultato = (double)a / b;    // 3.5
```

## Errore di indice (off-by-one)

```
int arr[5] = {1, 2, 3, 4, 5};  
for (int i = 0; i <= 5; i++) {    // ERRORE: i arriva a 5, ma arr[5] non  
    esiste!  
    cout << arr[i];  
}  
  
// Soluzione: usa < invece di <=  
for (int i = 0; i < 5; i++) {  
    cout << arr[i];  
}
```

**=** invece di **==** nelle condizioni

```
int x = 5;
if (x = 10) {    // ERRORE: assegna 10 a x, non confronta!
    cout << "uguale";
}

// Soluzione: usa == per confrontare
if (x == 10) {
    cout << "uguale";
}
```

## Ciclo infinito

```
int i = 0;
while (i < 10) {
    cout << i;
    // manca i++! il ciclo non finisce mai
}

// Soluzione: aggiorna sempre la variabile del ciclo
while (i < 10) {
    cout << i;
    i++;
}
```

## Il debugger in VS Code

VS Code ha un **debugger** integrato che ti permette di:

- Fermare il programma a una riga specifica (**breakpoint**)
- Vedere il valore di tutte le variabili in quel momento
- Eseguire il codice **una riga alla volta**

Come usarlo:

1. Clicca sul margine sinistro accanto al numero di riga per impostare un breakpoint (appare un punto rosso)
2. Premi **F5** per avviare il debug

3. Usa **F10** per eseguire riga per riga, **F11** per entrare dentro una funzione

## Strategia di debugging

Quando hai un bug, segui questi passi:

1. **Riproduci il bug:** assicurati di riuscire a farlo apparire in modo affidabile
2. **Isola il problema:** dove esattamente inizia ad andare storto? Aggiungi stampe per scoprirlo
3. **Formula un'ipotesi:** cosa pensi stia causando il bug?
4. **Testa l'ipotesi:** modifica il codice e verifica se il bug sparisce
5. **Verifica che non hai rotto altro:** testa anche gli altri casi

## Esempio: trovare un bug

```
// Programma con due bug: calcola la media di un array
#include <iostream>
using namespace std;

int main() {
    int voti[] = {8, 7, 9, 6, 10};
    int n = 5;
    int somma = 0;

    for (int i = 0; i <= n; i++) {    // BUG 1: <= invece di < → legge fuori
dall'array
        cout << "[DEBUG] i=" << i << ", voto=" << voti[i] << endl;
        somma += voti[i];
    }

    double media = somma / n;    // BUG 2: divisione intera → risultato
arrotondato
    cout << "Media: " << media << endl;
    return 0;
}

// Versione corretta:
// for (int i = 0; i < n; i++) {        ← usa < invece di <=
// double media = (double)somma / n;    ← converti prima di dividere
```

Con le stampe di debug, il bug 1 è visibile subito: quando `i=5`, `voti[5]` stampa un numero strano (non appartiene all'array).